



**AALBORG UNIVERSITY**

**MINI PROJECT  
ON  
“Electric Power Consumption Prediction using Machine  
Learning”**



**SUBMITTED BY:**

**SHASHANK UPADHAYAY**

**IMTIAJ AHMED**

**Innovative Communication Technologies**

**and**

**Entrepreneurship (ICTE)**

**2<sup>nd</sup> semester**

**SUBMITTED TO:**

**Hossam Farag**

## ABSTRACT

This project looks into how machine learning can help predict household electricity use to improve energy efficiency, cut costs, and support sustainable practices. Using a real dataset from the UCI Machine Learning Repository (covering 2006–2010), the study compares two popular machine learning methods: Linear Regression and Random Forest Regression. Before building the models, the data was carefully cleaned and prepared, including handling missing values and selecting useful features. Both models were trained using Python tools like Scikit-learn and evaluated based on how accurately they predicted power use. The evaluation used common error metrics like MAE, MSE, RMSE, and correlation scores. Results showed that Random Forest performed better than Linear Regression, especially in capturing complex patterns in the data. This shows that machine learning can play an important role in helping energy providers and planners make smarter decisions.

In addition to improving prediction accuracy, the project highlights the broader benefits of using machine learning in real-world applications. Understanding electricity usage patterns can help with smarter energy planning, reduce strain on power grids, and even lower environmental impact by promoting more efficient energy use. Looking ahead, the project could be enhanced by including other factors such as weather conditions, time of day, or real-time sensor data. More advanced models like deep learning could also be explored to improve the accuracy and reliability of forecasts, making this approach even more valuable for future energy systems.

# Contents

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 1   | Introduction.....                                                       | 1  |
| 1.1 | Background.....                                                         | 1  |
| 1.2 | Problem Statement .....                                                 | 1  |
| 1.3 | Objectives.....                                                         | 2  |
| 1.4 | Pertinence to Course Ideas.....                                         | 2  |
| 2   | Literature Review .....                                                 | 3  |
| 2.1 | Overview of Electric Power Consumption Prediction .....                 | 3  |
| 2.2 | Existing Methods and Tools.....                                         | 3  |
| 2.3 | Role of Machine Learning in Electric Power Consumption Prediction ..... | 4  |
| 3   | Dataset Description .....                                               | 4  |
| 3.1 | Source of the Dataset.....                                              | 4  |
| 3.2 | Attributes and Features .....                                           | 4  |
| 3.3 | Data Preprocessing.....                                                 | 5  |
| 4   | Methodology .....                                                       | 6  |
| 4.1 | Machine Learning Algorithm Used .....                                   | 6  |
|     | Linear Regression:.....                                                 | 6  |
|     | Random Forest Regression: .....                                         | 8  |
| 4.2 | Model Training and Evaluation.....                                      | 9  |
| 4.3 | Performance Metrics .....                                               | 9  |
| 5.  | Results and Analysis .....                                              | 11 |
| 6.  | Discussion.....                                                         | 12 |
| 6.1 | Challenges faced .....                                                  | 12 |
| 6.2 | Comparison with Expected Results .....                                  | 12 |
| 7.  | Conclusion .....                                                        | 12 |
| 8.  | Future Work .....                                                       | 13 |
|     | References .....                                                        | 14 |
| 9.  | Appendix.....                                                           | 15 |

# 1 Introduction

## 1.1 Background

Electric power consumption is one of the important topics of modern life that influence each and every field such as industries, household as well as businesses. Accurate power consumption prediction is required for the smart grid system to optimize energy efficiency so that forecasting of electricity usage can help in optimizing power generation, reducing costs as well as minimizing environmental hazards [1]. By leveraging trends and patterns machine learning techniques has become one of the powerful tools for predicting and analyzing electric power consumption.

In this mini-project, we will investigate the application of machine learning to predict electric power consumption. To create a prediction model for electric power consumption, WEKA is used to conduct the experiment using a variety of machine learning methods, including regression, supervised learning, and model evaluation.

## 1.2 Problem Statement

When it comes to power system planning, electric power demand forecasting is crucial. Thus, it is essential to create straightforward and efficient forecasting techniques [2]. Electricity is used in powering factories, homes, and businesses. The rise of its demands poses several challenges like resource management, cost optimization, and efficiency. Forecasting power consumption in an accurate manner is essential for efficient energy management, cost reduction as well as sustainable resource allocation. However, predicting power consumption is challenging because it depends on several factors like time, weather, power fluctuation as well as how people use it. Traditional techniques of power estimation often fail or are poor to capture these complex dependencies to inefficiencies in energy distribution as well as planning. The goal of this mini project is for the analyze of electric power usage using machine learning techniques in python which helps us to find hidden patterns as well as knowledge that can contribute to effective energy management and sustainability.

Key Questions Addressing the Problem:

- How can a machine learning platform like Python help to analyze electric power consumption trends?

- How can the ideas generated by this project contribute to energy efficiency, cost reduction, as well as sustainable power consumption?

### 1.3 Objectives

The objectives of this mini-project are:

- Analyzing electric power consumption data using Python.
- Preprocessing the dataset as well as exploring
- Applying a machine learning algorithm for prediction and classification.
- Comparing the performance of different models like regression, decision tree, and finding the best one
- Finding appropriate improvements that can be made in energy consumption and management

### 1.4 Pertinence to Course Ideas

The mini project aligns with important machine learning concepts covered in our course, which are as follows:

**Supervised Learning:** A machine learning method called supervised learning trains artificial intelligence algorithm models using labeled datasets to find the underlying patterns and connections between input features and outputs. Here, building a model that can accurately forecast results on fresh real-world data is the goal of the learning process [3].

**Regression Analysis:** It is used for predicting outcomes that are continuous. Here, models try to predict the target output in regression issues, when the output is a continuous value. Financial planning and sales revenue estimates are examples of regression jobs. Regression methods include linear regression, logistic regression, and polynomial regression. Data analysts can use these methods to gain a better understanding of the relationships between variables, make predictions, and unravel complex patterns in data [4].

**Linear Regression:** A supervised learning algorithm utilized for predictive modeling is linear regression, and it determines the connection between dependent and independent

variables by aligning a straight line. Predictive modeling in data science can be effectively utilized by linear regression. Though it works well for linear relationships, it has limitations when dealing with complex patterns. [5]

**Random forest:** It is a flexible supervised machine learning algorithm used for both classification as well as regression purposes. It combines the output of multiple decision trees to reach a single result. The algorithm's strength is its capacity to manage intricate datasets and reduce overfitting, which makes it an effective tool for a range of machine learning predictive tasks [6].

Data scientists can use dimensionality reduction to simplify the complexity of huge datasets, which often contain a lot of characteristics. By limiting the number of features to those that are most important for predicting data labels, this data science technique maintains accuracy while boosting efficiency [3].

## 2 Literature Review

### 2.1 Overview of Electric Power Consumption Prediction

For decades, the field of computer architecture has conducted extensive studies on energy consumption. The use of energy as a metric in machine learning is becoming more common, but most research continues to prioritize achieving high accuracy without regard for computational limitations [7]. It is important to predict electric power consumption in order to ensure efficient energy use, manage costs, and maintain system stability. In addition, it is essential to predict electric power consumption to assist the electrical power industry in its planning and to ensure the proper functioning of electrical power systems, as this kind of energy cannot be stored; precise predictions are essential for maintaining a balance between the supply of electricity and demand [8].

### 2.2 Existing Methods and Tools

A variety of tools and methods have been created to forecast power consumption. Random forests, regression, gradient boosting, etc, are examples of machine learning models that have demonstrated enhanced precision and forecasting electric power consumption [9].

Visual Studio Code integrates the flexibility of a source code editor with robust developer tools, such as IntelliSense for code completion as well as debugging, and it is one of the most popular and lightweight code editors for Python development. [10]

Python is a high-level object-oriented programming language, and its clean syntax and extensive frameworks make it easy to write and understand machine learning algorithms. Python libraries such as scikit-learn, TensorFlow, and XGBoost offer scalability and model customization. It supports program modularity as well as the reuse of code. [11]

### 2.3 Role of Machine Learning in Electric Power Consumption Prediction

Machine learning is becoming more important in energy forecasting by facilitating accurate, automated modeling in real time, and it assists the government in establishing expectations, accurate forecasts to arrange its budget more effectively, and supports economic agents in their decision-making [8]. They continuously learn from new data and enhance their predictions. Methods like random forest contribute to enhancing the robustness and precision of these models. Moreover, by training machine learning models to adapt to particular datasets and comprehend the underlying patterns within them, tasks such as forecasting, fraud detection, and risk assessment can become more precise [12].

## 3 Dataset Description

### 3.1 Source of the Dataset

The dataset that is being used for this mini-project was sourced from the UCI Machine Learning Repository, a renowned platform for standardized datasets employed in machine learning research and education, and the dataset specifically relates to household electric power consumption, collected over four years which is from 2006 to 2010. It includes thorough measurements of power consumption, as well as the related temporal and environmental factors, and finally, this dataset serves as a dependable and realistic foundation for the creation and assessment of predictive models dealing with electric power consumption. [13]

### 3.2 Attributes and Features

The dataset includes the following key attributes relevant to forecasting electric power consumption:

- Date and Time: They represent the timestamp of each observation and are later merged as well converted into a datetime format to support time-series analysis.
- Global active power (kW): The primary variable of interest here is the total active power (in kilowatts) used by the household.
- Global reactive power (kW): The reactive power consumed (non-working power), also in kilowatts.
- Voltage (V): The average voltage over the period, which might be linked to power consumption patterns.
- Global intensity (A), which is the total current intensity (in amperes).
- Sub\_metering\_1, Sub\_metering\_2, Sub\_metering\_3, which represent energy consumption recorded by individual household sub-meters. For example, sub\_metering\_1 refers to the kitchen, sub\_metering\_2 to the laundry room, etc.

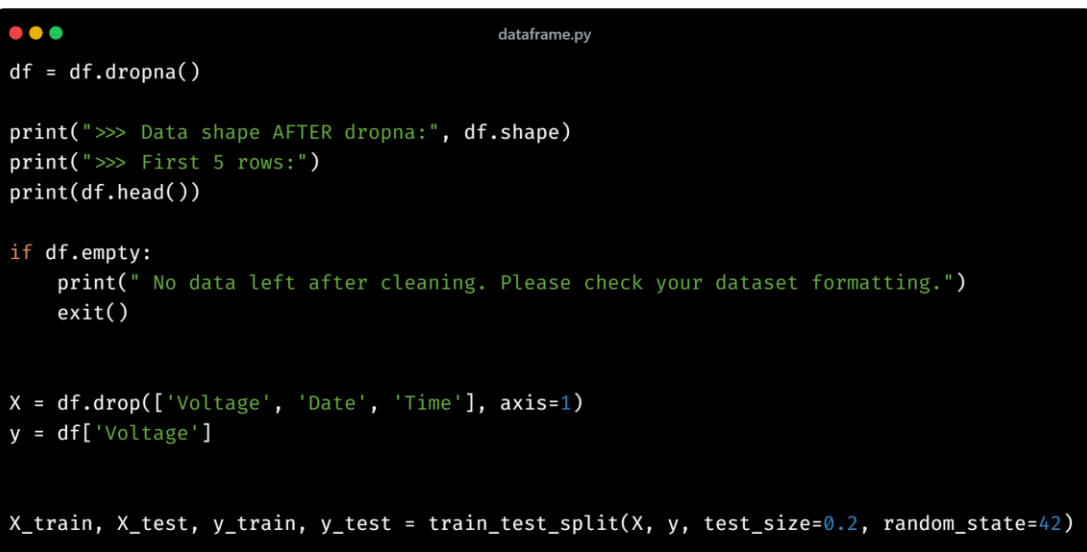
### 3.3 Data Preprocessing

Data preprocessing (DP) is a crucial phase of ML applications, and it has been reported to significantly affect the performance of ML methods [14]. It consists of assessing, filtering, manipulating, and encoding data to make it comprehensible for a machine learning algorithm and usable for generating output. The primary goal and aim of data preprocessing is to resolve data problems like noise and outliers, missing values, enhance data quality, and prepare the data for machine learning applications [15].

The following steps were done in Python to clean and prepare the electric power consumption dataset, that are listed below:

- The dataset was initially stored in a text file using ; as a delimiter, so it was manually parsed and loaded into a Pandas data frame
- Missing values were handled, and all rows containing NaN values in any of the key numeric columns were removed. `df = df.dropna()` is used, which ensures complete data points are used for training, resulting in improved quality of the model.

- Several important features, like Global active power, Voltage, and Submetering, were initially represented in string format. So, these were changed into numerical values.
- Feature selection and target definition is effectively organized. Features are separated from the target variable, and Voltage is chosen as the target, and the input features exclude non-numeric columns like data and time.
- The dataset is split into 80% training and 20% testing, that helps proper model evaluation.



```
dataframe.py
df = df.dropna()

print(">>> Data shape AFTER dropna:", df.shape)
print(">>> First 5 rows:")
print(df.head())

if df.empty:
    print(" No data left after cleaning. Please check your dataset formatting.")
    exit()

X = df.drop(['Voltage', 'Date', 'Time'], axis=1)
y = df['Voltage']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

*Figure 1 Data Preprocessing*

## 4 Methodology

### 4.1 Machine Learning Algorithm Used

#### Linear Regression:

Linear regression estimates the value of a dependent variable based on an independent variable. It fits a linear line to best represent the relationship between the dependent and independent variables based on numerous points provided to the model, which is similar to a scatter plot, and also operates by generating a linear equation ( $y = mx + c$ ) that best predicts the values of dependent variables, solving for  $m$  (slope) and  $c$  (y-intercept).

**Cost Function:** A cost function (or Loss Function) is one of the important for directing models toward accurate predictions, as it quantifies the deviation of the predicted output from the actual value.

The formula for the cost function is shown as below:

$$J(\theta) = 1/2m \sum_{\{i=1\}}^{\{m\}} \left( h_{\theta}(x^{\{i\}}) - y^{\{i\}} \right)^2$$

Here,

$J(\theta)$  is the cost function

$m$  is the number of training examples

$h_{\theta}(x^{(i)})$  is the predicted value based on model parameters  $\theta$ .

$y(i)$  is the actual output for the  $i$ (th) input

$\theta$  is the parameters/weights of the given model

For regression, cost functions like Mean Squared Error (MSE) and Mean Absolute Error (MAE) are commonly used.

The screenshot shows a VS Code editor with a Python script in a file named `new.py`. The script defines a function `print_full_metrics` that calculates and prints various performance metrics for a given model, true values, and predicted values. The script then uses `LinearRegression` and `RandomForestRegressor` to fit models on training data and predict on test data. The terminal output shows the results for both models.

```

50
51
52 lr = LinearRegression()
53 lr.fit(X_train, y_train)
54 y_pred_lr = lr.predict(X_test)
55
56
57 rf = RandomForestRegressor(n_estimators=100, random_state=42)
58 rf.fit(X_train, y_train)
59 y_pred_rf = rf.predict(X_test)
60
61
62 def print_full_metrics(name, y_true, y_pred):
63     print(f"\n--- {name} ---")
64     mae = mean_absolute_error(y_true, y_pred)
65     mse = mean_squared_error(y_true, y_pred)
66     rmse = np.sqrt(mse)
67     r = np.corrcoef(y_true.astype(float), y_pred.astype(float))[0, 1]
68     mean_actual = np.mean(y_true.astype(float))
69
70
71 --- Linear Regression ---
72 MAE: 2.1257
73 MSE: 7.9154
74 RMSE: 2.8134
75 R: 0.4978
76 RAE (%): 85.38
77 RRSE (%): 86.73
78
79 --- Random Forest ---
80 MAE: 1.6235
81 MSE: 4.9917
82 RMSE: 2.2342

```

Figure 2 Linear Regression with Performance Metrics Output

### Random Forest Regression:

The random forest algorithm combines the results of many decision trees to generate one outcome in order to reach a single result. It manages data sets that include both continuous variables, as seen in regression, as well as categorical variables, as seen in classification. Thus, its performance is superior for classification and regression tasks. It manages the overfitting (when a model picks up not just the fundamental trends in the training data, but also the noise and random variations) problem by the randomness in data samples and feature selection that makes it an important tool for predictive work in machine learning [18].

### Steps for Random Forest Algorithm:

Step 1: It constructs numerous decision trees based on random data samples, and a different subset of the data is used to train each tree, resulting in the uniqueness of each tree.

Step 2: The algorithm picks a random subset of features or variables to split the data when creating each tree, rather than using all available features at once, as a result, it enhances the variety of trees.

Step 3: Each and every decision tree in the forest generates a prediction using the data it was trained on, and finally, the random forest determines the final result by aggregating the predictions from all the trees.

The majority vote helps to identify the final prediction for classification, and for regression, the final prediction is the average of prediction for each tree in the forest.

```

64
65 def print_full_metrics(name, y_true, y_pred):
66     print(f"\n--- {name} ---")
67     mae = mean_absolute_error(y_true, y_pred)
68     mse = mean_squared_error(y_true, y_pred)
69     rmse = np.sqrt(mse)
70     r = np.corrcoef(y_true.astype(float), y_pred.astype(float))[0, 1]
71     mean_actual = np.mean(y_true.astype(float))
72     rae = np.sum(np.abs(y_pred - y_true)) / np.sum(np.abs(mean_actual - y_true)) * 100
73     rrse = (rmse / np.sqrt(np.mean((y_true - mean_actual) ** 2))) * 100
74
75     print("MAE: ", round(mae, 4))
76     print("MSE: ", round(mse, 4))
77     print("RMSE: ", round(rmse, 4))
78     print("R: ", round(r, 4))
79     print("RAE (%) : ", round(rae, 2))
80     print("RRSE (%) : ", round(rrse, 2))
81
82 print_full_metrics("Linear Regression", y_test, y_pred_lr)
83 print_full_metrics("Random Forest", y_test, y_pred_rf)

```

```

--- Random Forest ---
MAE: 1.6235
MSE: 4.9917
RMSE: 2.2342
R: 0.7284
RAE (%): 65.21
RRSE (%): 68.87
PS C:\Users\Shashank\OneDrive\Desktop\project_electric> & C:/Users/Shashank/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Shashank/OneDrive/Desktop/project_electric/new.py
>>> RUNNING CLEAN VERSION
>>> Data shape BEFORE dropna: (2075259, 9)

```

Figure 3 Random Forest with Performance Metrics Output

## 4.2 Model Training and Evaluation

By using the Python platform, models were trained with an 80-20 train-test split, and preprocessing involved normalization as well as encoding. Scikit-learn is used to implement Linear Regression and Random Forest, and the test set was used to evaluate model performance using metrics such as MAE, MSE, RMSE, correlation coefficient, relative absolute error, and root relative squared error.

## 4.3 Performance Metrics

- Mean Absolute Error (MAE): It measures the absolute difference between actual and predicted values.

$$MAE = 1/n \sum_{i=1}^n |y_i - \hat{y}_i|$$

**Where:**

N is the number of observations

Y<sub>i</sub> is the actual value

$y^i$  is the predicted value

$|y_i - y^i|$  is the absolute error for every observation

- Mean Squared Error (MSE): It is the average squared difference between the actual values and the predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - y^i)^2$$

- Root Mean Squared Error (RMSE): It is defined as the square root of the Mean Squared Error.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - y^i)^2}$$

- Correlation coefficient (R): It measures the linear relationship between actual values and predicted values.

$$r = \frac{\sum_{i=1}^n (y_i - \bar{y})(y^i - \bar{y^i})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2} \cdot \sqrt{\sum_{i=1}^n (y^i - \bar{y^i})^2}}$$

where:

- $y_i$  is the actual value  
 $y^i$  is the predicted value  
 $\bar{y}$  is the mean of actual values  
 $\bar{y^i}$  is the mean of predicted values  
 $n$  is the number of observation
- Relative Absolute Error (%): It measures how far off predictions are from the actual values  
If  $RAE < 100\%$  the model is better than the mean predictor

If  $RAE > 100\%$  the model is worse than just predicting the mean

- Root Relative Squared Error (%): It is normalized by the performance of a basic model that consistently predicts the average of the target variable.

If  $RRSE = 0\%$  it is the perfect predictions

If  $RRSE = 100\%$  it is same as using the mean

If  $RRSE > 100\%$  it is worse than mean prediction

## 5. Results and Analysis

Based on the results of WEKA as well as Python, the comparison table between Linear Regression and Random Forest is shown below:

| <b>Metrics</b>                  | <b>Linear Regression</b> | <b>Random Forest</b> |
|---------------------------------|--------------------------|----------------------|
| Mean Absolute Error (MAE)       | 2.1257                   | 1.6235               |
| Mean Squared Error (MSE)        | 7.9154                   | 4.9917               |
| Root Mean Squared Error (RMSE)  | 2.8134                   | 2.2342               |
| Correlation Coefficient (R)     | 0.4987                   | 0.7284               |
| Relative Absolute Error (%)     | 85.38                    | 65.21                |
| Root Relative Squared Error (%) | 86.73                    | 68.87                |

From the above table, it is clear that Random Forest significantly outperforms Linear Regression across all metrics. The correlation coefficient is higher in Random Forest than linear regression, i.e. 0.7284, indicating stronger predictive power. Both MAE and RMSE are lower in the Random Forest model, showing better accuracy.

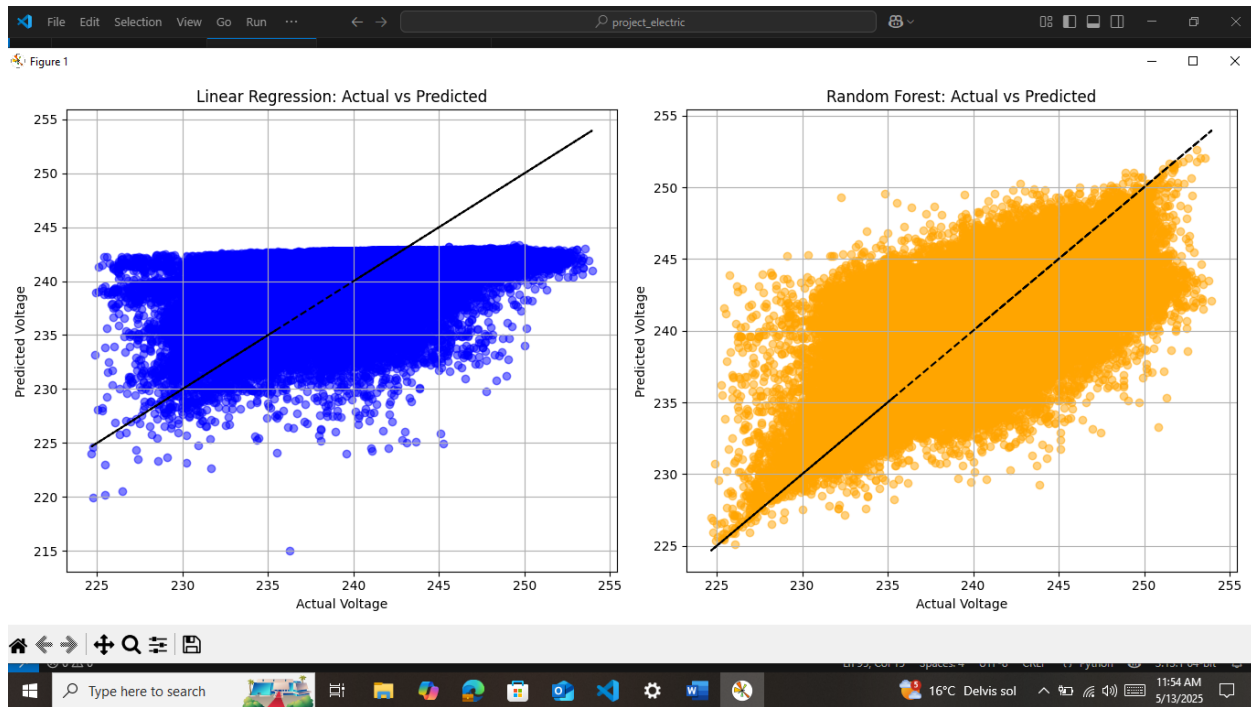


Figure 4 Visual Graph comparing Predicted vs Actual Voltage

## 6. Discussion

### 6.1 Challenges faced

**The problems that were faced while doing the project are listed below:**

- Handling a large dataset, as well as handling missing data, was difficult.
- To run a random forest with a large dataset was computationally intensive as well as time-consuming.
- Balancing theory and background research and result analysis within a limited timeframe was a little bit challenging.

### 6.2 Comparison with Expected Results

Random Forest maintained up to expectations by demonstrating a strong performance whereas linear regression was limited by its assumption of linearity.

## 7. Conclusion

The project highlights the use of machine learning algorithms such as linear regression and random forest for predicting electric power consumption. It also addresses challenges of accurately forecasting electric power consumption, which can be essential for smart grid systems and cost efficiency. By using the dataset from the UCI repository, data preprocessing to clean the dataset

was performed, and the result shows that the random forest technique outperforms linear regression by comparing performance metrics such as root mean squared error and correlation coefficient. Thus, this shows the effectiveness of applied methods in identifying and solving complex datasets for accurate prediction and forecasting.

The research question that was addressed in the problem statement: How can a machine learning platform like Python help to analyze electric power consumption trends? To answer this question, Python libraries were used, such as Pandas, Scikit-learn, etc., that helped in preprocessing, model evaluation, as well as accurate prediction that shows the power capability of Python to handle real-world energy consumption datasets. To answer the second question, how can the ideas generated by this project contribute to energy efficiency, cost reduction, as well as sustainable power consumption? The technique to forecast electric power consumption accurately helps energy providers in the proper distribution and management of energy, which helps to control overproduction, ultimately leading to optimized energy resource usage.

## 8. Future Work

Future enhancements can include:

- External factors, such as weather data, can be included to improve prediction.
- Real-time prediction dashboard can be developed, and deep learning can be applied for time series forecasting can be done

## References

- [1] J. P. E. H. & S. J. Jihoon Moon, "Forecasting power consumption for higher educational institutions based on machine learning," *The Journal of Super Computing*, vol. 74, pp. 3778-3800, 29 March 2017.
- [2] I. R. K. & A. S. S. Chetan B. Khadse, Investigation of Electric Load Forecasting Methods: A Weka Application (Regression and Optimization), 2024, pp. 69-84.
- [3] I. B. & C. Stryker, "IBM," [Online]. Available: <https://www.ibm.com/think/topics/supervised-learning>.
- [4] L. Yen, 07 September 2023. [Online]. Available: <https://www.datamation.com/big-data/what-is-regression-analysis/>.
- [5] [Online]. Available: <https://decodingdatascience.com/linear-regression-a-complete-guide-with-examples/>.
- [6] Sruthi, 11 December 2024. [Online]. Available: <https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/>.
- [7] C. F. R. R. , G. Eva García-Martín, "Estimation of energy consumption in machine learning," *Journal of Parallel and Distributed Computing*, 21 August 2019.
- [8] D. O. C. M. D. R. Pedro C. Albuquerque, "Machine learning models for forecasting power electricity consumption using a high dimensional dataset," *Expert Systems with Applications*, 24 September 2021.
- [9] V. R. C. C. C. E. S. C. O. B. Cristina Bianca Pop, "Forecasting the Short-Term Energy Consumption".
- [10] [Online]. Available: <https://code.visualstudio.com/Docs/editor/whyvscode>.  
]
- [11] [Online]. Available: <https://www.python.org/doc/essays/blurb/>.  
]
- [12] T. Jądzewski, 04 March 2025. [Online]. Available: <https://virtuslab.com/blog/data/benefits-and-advantages-of-machine-learning/>.  
]
- [13] [Online]. Available: <https://archive.ics.uci.edu/datasets/?search=Individual+Household+Electric+Power+Consumption>.
- [14] , Y.-F. L. b. M. X. Jianglin Huang, "An empirical analysis of data preprocessing for machine learning-based software cost estimation," *Information and Software Technology*, vol. 67, pp. 108-127, November 2015.
- [15] I. Novogroder, 30 April 2024. [Online]. Available: <https://lakefs.io/blog/data-preprocessing-in-machine-learning/>.  
]

- [16 J. R. Ballesteros, 22 October 2023\]. [Online]. Available:  
] <https://medium.com/@jrballesteros/weka-how-to-learn-machine-learning-for-non-experts-70e9767b08b2>.
- [17 "Introduction to WEKA".  
]
- [18 Sruthi, 01 May 2025. [Online]. Available:  
] <https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/>.
- [19 [Online]. Available: <https://colab.research.google.com/>.  
]

## 9. Appendix

Google Collaboratory, or Colab, is a hosted Jupyter Notebook service that offers free access to computing resources, including GPUs and TPUs, without requiring any setup to use. Colab is particularly ideal for machine learning, data science, and educational purposes [19].

WEKA, a robust open-source data mining software, is extensively utilized in educational and research settings. It offers convenient access to a collection of ML algorithms for classification, regression, clustering, and additional tasks [17].

### Weka Platform Overview

It's an independent open-source software that offers data preprocessing tools, implements machine Learning algorithms, and provides visualization capabilities, enabling users to create machine learning techniques and apply them to real-world data challenges. The following diagram provides a summary of the tools. The University of Waikato in New Zealand runs the Weka initiative [16]. The platform is a user-friendly tool that offers a variety of well-suited algorithms. It offers tools for data pre-processing, classification, regression, clustering, association rules, and visualization. It is also ideal for creating new machine learning schemes [17].

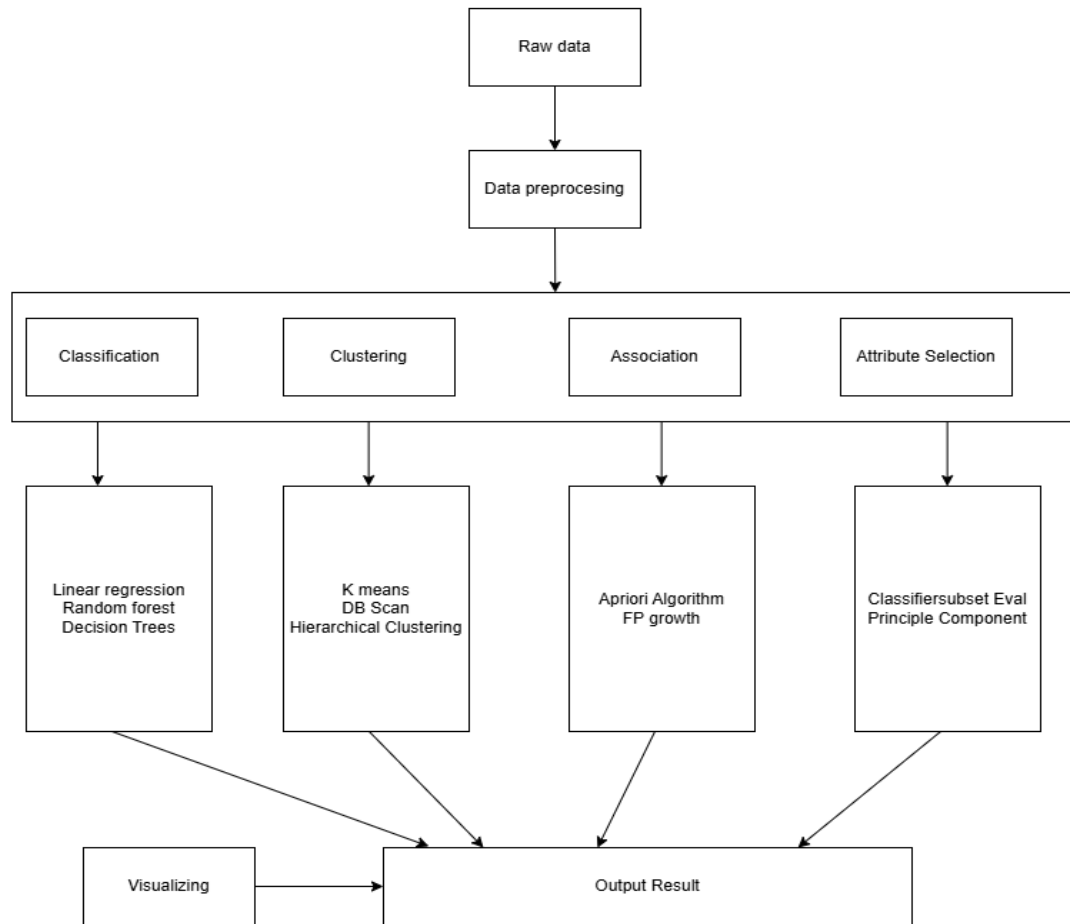


FIG: About WEKA. [17]

In WEKA, two types of files can be used: CSV and other files. ARFF

```

dataframe.py

import csv
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt

print(">>> RUNNING CLEAN VERSION")

data = []
with open('power_consumption.txt', 'r') as file:
    reader = csv.reader(file, delimiter=';')
    for row in reader:
        data.append(row)

df = pd.DataFrame(data)
df.columns = df.iloc[0]
df = df[1:]

numeric_cols = ['Global_active_power', 'Global_reactive_power', 'Voltage',
                 'Global_intensity', 'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3']

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

print(">>> Data shape BEFORE dropna:", df.shape)
print(">>> NaNs per numeric column:")
print(df[numeric_cols].isna().sum())

df = df.dropna()

print(">>> Data shape AFTER dropna:", df.shape)
print(">>> First 5 rows:")
print(df.head())

if df.empty:
    print(" No data left after cleaning. Please check your dataset formatting.")
    exit()

X = df.drop(['Voltage', 'Date', 'Time'], axis=1)
y = df['Voltage']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

```

```

dataframe.py

def print_full_metrics(name, y_true, y_pred):
    print(f"\n--- {name} ---")
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r = np.corrcoef(y_true.astype(float), y_pred.astype(float))[0, 1]

    mean_actual = np.mean(y_true.astype(float))
    rae_denom = np.sum(np.abs(mean_actual - y_true))
    rrse_denom = np.sqrt(np.mean((y_true - mean_actual) ** 2))

    try:
        rae = np.sum(np.abs(y_pred - y_true)) / rae_denom * 100 if rae_denom != 0 else float('nan')
    except Exception as e:
        print(f"RAE Error: {e}")
        rae = float('nan')

    try:
        rrse = (rmse / rrse_denom) * 100 if rrse_denom != 0 else float('nan')
    except Exception as e:
        print(f"RRSE Error: {e}")
        rrse = float('nan')

    print("MAE:      ", round(mae, 4))
    print("MSE:      ", round(mse, 4))
    print("RMSE:      ", round(rmse, 4))
    print("R:         ", round(r, 4))
    print("RAE (%):   ", round(rae, 2) if not np.isnan(rae) else "NaN")
    print("RRSE (%): ", round(rrse, 2) if not np.isnan(rrse) else "NaN")

print_full_metrics("Linear Regression", y_test, y_pred_lr)
print_full_metrics("Random Forest", y_test, y_pred_rf)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_lr, alpha=0.5, color='blue')
plt.plot(y_test, y_test, color='black', linestyle='--')
plt.xlabel('Actual Voltage')
plt.ylabel('Predicted Voltage')
plt.title('Linear Regression: Actual vs Predicted')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_rf, alpha=0.5, color='orange')
plt.plot(y_test, y_test, color='black', linestyle='--')
plt.xlabel('Actual Voltage')
plt.ylabel('Predicted Voltage')
plt.title('Random Forest: Actual vs Predicted')
plt.grid(True)

plt.tight_layout()
plt.savefig("model_comparison_voltage.png", dpi=300)
plt.show()

```

Figure 5 Python Program